

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

Інститут комп'ютерних систем

Кафедра інженерії програмного забезпечення

Ілля ВЕРБАНОВ

(група АС-222)

КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА
Вебзастосунок для пошуку загублених домашніх тварин

Спеціальність:

121 Інженерія програмного забезпечення

Освітньо-професійна програма:

Інженерія програмного забезпечення

Керівник:

Надія БЕГЛОВА, асистент каф. ІПЗ

Одеса – 2026

АНОТАЦІЯ

Вербанов І. Г. Вебзастосунок для пошуку загублених домашніх тварин : кваліфікаційна робота бакалавра за спеціальністю «121 Інженерія програмного забезпечення» / Ілля Геннадійович Вербанов ; керівник Надія Миколаївна Беглова – Одеса : Нац. ун-т «Одес. політехніка», 2026. – 59 с.

Кваліфікаційна робота містить основну текстову частину на 48 сторінках, список використаних джерел з 13 найменувань на 1 сторінці, додатки на 4 сторінках.

Метою кваліфікаційної роботи є скорочення часу возз'єднання загублених домашніх тварин з власниками шляхом автоматизованого зіставлення оголошень про втрату та про знахідку за видом тварини, географічною відстанню та часовим вікном події.

У роботі розроблено вебзастосунок, який дозволяє власникам публікувати оголошення про загублену тварину, а особам, які знайшли тварину, — оголошення про знахідку. Реалізовано алгоритм формування ранжованого переліку оголошень-кандидатів, що оцінює збіг за видом тварини, відстанню між місцями події за формулою гаверсинуса та різницею дат. Запропонований кандидат підтверджується людиною: після підтвердження зв'язку контактні дані сторін взаємно розкриваються, що дозволяє організувати повернення тварини.

Систему реалізовано із застосуванням платформи Node.js, фреймворку NestJS, бібліотеки React, ORM Drizzle та системи керування базами даних PostgreSQL. У процесі розробки проведено аналіз предметної області та аналогів, сформовано функціональні та нефункціональні вимоги, виконано планування проекту, спроектовано архітектуру системи, структуру бази даних і користувацький інтерфейс, проведено функціональне та нефункціональне тестування.

Результатом кваліфікаційної роботи є вебзастосунок для пошуку загублених тварин, який може бути використаний волонтерськими спільнотами та притулками для автоматизації первинного зіставлення оголошень. Проведене тестування підтвердило коректність роботи системи та її відповідність поставленим вимогам.

Ключові слова: вебзастосунок, пошук загублених тварин, алгоритм зіставлення, формула гаверсинуса, ранжування кандидатів, база даних, NestJS, React.

ABSTRACT

Verbanov I. H. Web application for finding lost pets : Bachelor's qualification work in the specialty «121 Software Engineering» / Illia Hennadiiovych Verbanov ; supervisor Nadiia Mykolaivna Behlova – Odesa : Odesa Polytechnic Nat. Univ., 2026. – 59 pages.

The qualification work contains the main text part on 48 pages, a list of used sources with 13 titles on 1 page, and appendices on 4 pages.

The aim of the qualification work is to reduce the time of reuniting lost pets with their owners through automated matching of lost and found reports by animal species, geographic distance, and the time window of the event.

The work presents a web application that allows owners to publish reports about a lost pet and finders to publish reports about a found animal. A matching algorithm is implemented that builds a ranked list of candidate reports, evaluating the match by species equality, the distance between event locations computed with the haversine formula, and the difference of dates. A proposed candidate is confirmed by a human: after the link is confirmed, the contact details of both parties are mutually revealed so that the return of the pet can be arranged.

The system is implemented using the Node.js platform, the NestJS framework, the React library, the Drizzle ORM, and the PostgreSQL database management system. During the development, the subject area and existing solutions were analyzed, functional and non-functional requirements were defined, project planning was carried out, the system architecture, database structure, and user interface were designed, and functional and non-functional testing was performed.

The result of the qualification work is a web application for finding lost pets that can be used by volunteer communities and shelters to automate the initial matching of reports. Testing confirmed the correctness of the system and its compliance with the specified requirements.

Keywords: web application, lost pet search, matching algorithm, haversine formula, candidate ranking, database, NestJS, React.

ЗМІСТ

Вступ	7
1 Специфікація вимог до вебзастосунку	10
1.1 Опис предметної області	10
1.2 Аналіз аналогів	11
1.3 Функціональні вимоги	15
1.4 Нефункціональні вимоги	16
1.5 Вимоги до інтерфейсу користувача	17
1.6 Варіанти використання системи	17
Висновки до розділу 1	20
2 Планування проєкту з розробки вебзастосунку	21
2.1 Оцінка масштабів проєкту	21
2.2 Оцінка ризиків	22
Висновки до розділу 2	24
3 Алгоритм зіставлення оголошень про загублених і знайдених тварин	25
3.1 Основні положення	25
3.2 Формування ознак оголошення	25
3.3 Алгоритм попередньої фільтрації кандидатів	26
3.4 Алгоритм оцінювання відповідності оголошень	26
3.5 Алгоритм ранжування кандидатів	27
Висновки до розділу 3	29
4 Проєктування вебзастосунку	31
4.1 Проєктування моделі оголошення та зв'язку	31
4.2 Архітектура вебзастосунку	31
4.3 Проєктування структури бази даних	33
Висновки до розділу 4	36
5 Програмна реалізація вебзастосунку	37
5.1 Вибір інструментів розробки	37
5.2 Реалізація сервісу зіставлення оголошень	38
Висновки до розділу 5	39
6 Тестування вебзастосунку	40
6.1 Тестування функціональних вимог	40

6.2 Тестування нефункціональних вимог	42
Висновки до розділу 6	42
7 Експлуатація застосунку	44
7.1 Робота користувача з системою	44
7.2 Приклад зіставлення оголошень про загублену тварину	49
7.3 Розгортання застосунку адміністратором (розробником)	50
Висновки до розділу 7	52
Загальні висновки	53
Список використаних джерел	55
Додаток А Програмна реалізація схеми бази даних	56
Додаток Б Програмна реалізація сервісу зіставлення оголошень	57
Додаток В Програмна реалізація обчислення відстані за формулою гаверсинуса	58
Додаток Г Програмна реалізація сервісу підтвердження зв'язку	59

ВСТУП

Актуальність теми роботи. Втрата домашньої тварини є поширеною ситуацією, у якій вирішальне значення має час: чим довше тварина залишається ненайденою, тим менша ймовірність її повернення. За даними міжнародних досліджень (ASPCA, American Humane Association), протягом життя власника втрачає близько 14–15 % собак і близько 15 % котів; із них своїми силами або через сусідів повертаються близько 60 % собак, тоді як для котів цей показник не перевищує 30 %. У межах першого тижня після зникнення додому повертається лише близько третини тварин, після чого ймовірність повернення стрімко знижується. В умовах України ситуація додатково ускладнюється воєнним станом і вимушеною міграцією населення: за оцінками громадських організацій, що опікуються тваринами (UAnimals, «Сіріус», «Happy Paw»), кількість загублених і покинутих домашніх тварин у великих містах зросла у 2–3 рази порівняно з довоєнним періодом, а основними каналами пошуку залишаються неструктуровані дописи у Telegram-каналах, Facebook-групах і на класифайдах на кшталт OLX. Власник, який втратив тварину, і людина, яка її знайшла, зазвичай діють розрізнено: оголошення розміщуються в різних соціальних мережах, тематичних групах і дошках оголошень, не мають єдиної структури та не зіставляються між собою автоматично. Зіставлення оголошення про втрату з оголошенням про знахідку виконується людьми вручну, спирається на випадковий перегляд стрічки і значною мірою залежить від того, чи побачать обидві сторони допис одна одної. Це збільшує час пошуку і знижує шанси на возз'єднання тварини з власником.

Практична цінність розробленого вебзастосунку полягає в тому, що він замінює неструктурований ручний пошук на автоматичне геопросторове та видо-орієнтоване зіставлення оголошень: одразу після публікації оголошення про знахідку власник загубленої тварини у тій самій географічній зоні отримує ранжований перелік ймовірних збігів без необхідності переглядати десятки сторонніх дописів. Цільовою аудиторією є власники домашніх тварин, волонтери зоозахисних організацій та небайдужі громадяни, які знайшли тварину. Очікуваним практичним ефектом є скорочення середнього часу від зникнення до возз'єднання та підвищення частки повернутих тварин, передусім за рахунок

раннього виявлення збігу у перші 24–72 години — найкритичніший проміжок часу для пошуку.

Актуальність роботи зумовлена такими факторами:

- поширеністю випадків втрати домашніх тварин і високою соціальною значущістю їх повернення;
- розрізненістю наявних каналів публікації оголошень і відсутністю єдиної структурованої моделі оголошення;
- необхідністю скорочення часу пошуку за рахунок автоматичного зіставлення оголошень про втрату та про знахідку;
- потребою у захисті персональних контактних даних до моменту підтвердження збігу;
- розвитком вебтехнологій, які дозволяють реалізувати геопросторове зіставлення без спеціалізованих картографічних розширень бази даних.

Темою кваліфікаційної роботи є розробка вебзастосунку, що забезпечує публікацію структурованих оголошень про загублених і знайдених домашніх тварин та автоматичне формування переліку ймовірних збігів між ними.

Мета і задачі роботи. Метою роботи є скорочення часу возз'єднання загублених домашніх тварин з власниками за рахунок автоматизації первинного зіставлення оголошень про втрату та про знахідку. Для досягнення поставленої мети необхідно вирішити такі задачі:

- дослідити предметну область пошуку загублених домашніх тварин;
- провести аналіз існуючих рішень і підходів;
- сформулювати функціональні та нефункціональні вимоги до системи;
- виконати планування проєкту та оцінювання ризиків;
- розробити алгоритм зіставлення та ранжування оголошень-кандидатів;
- спроєктувати архітектуру застосунку, структуру бази даних і користувацький інтерфейс;
- реалізувати вебзастосунок із використанням сучасних технологій;
- провести тестування функціональних і нефункціональних вимог системи.

Об'єкт роботи. Об'єктом роботи є процес пошуку та повернення загублених домашніх тварин на основі зіставлення оголошень про втрату та про знахідку.

Предмет роботи. Предметом роботи є моделі, алгоритм і програмні засоби вебзастосунку, що забезпечують структуроване подання оголошень і автоматичне формування ранжованого переліку ймовірних збігів із подальшим підтвердженням зв'язку людиною.

1 СПЕЦИФІКАЦІЯ ВИМОГ ДО ВЕБЗАСТОСУНКУ

1.1 Опис предметної області

Втрата домашньої тварини зачіпає значну частину власників протягом життя тварини. Успіх повернення залежить від того, наскільки швидко інформація про втрату досягне людини, яка тварину знайшла, і навпаки. Дослідження практик повернення тварин показують, що більшість успішних випадків припадає на перші дні після зникнення та на невелику відстань від місця, де тварину востаннє бачили [1].

Сьогодні власник, який втратив тварину, найчастіше діє так: розміщує допис у місцевих групах соціальних мереж, на дошках оголошень, звертається до притулків і ветеринарних клінік. Людина, яка знайшла тварину, діє симетрично, але незалежно. Оголошення обох сторін потрапляють у різні канали, мають довільну структуру і не зіставляються між собою. Збіг виявляється випадково — якщо обидві сторони переглянуть допис одна одної у стрічці.

Така організація процесу має кілька недоліків. По-перше, оголошення не структуровані: вид тварини, місце та дата події часто наведені у вільній формі, що ускладнює пошук. По-друге, зіставлення виконується вручну і залежить від обсягу стрічки та уважності читача. По-третє, контактні дані публікуються відкрито, що створює ризик зловживань ще до того, як збіг підтверджено.

У наукових джерелах і практичних посібниках наголошується, що ключовими ознаками для зіставлення є вид тварини, географічна близькість місць події та невеликий проміжок часу між втратою і знахідкою [1, 2]. Саме ці ознаки піддаються формалізації та автоматичному порівнянню.

Таким чином, процес пошуку загублених тварин потребує єдиної структурованої моделі оголошення та автоматичного зіставлення оголошень про втрату і про знахідку з подальшим підтвердженням збігу людиною.

1.2 Аналіз аналогів

Аналіз наявних рішень дозволяє виявити вдалі підходи до побудови систем пошуку загублених тварин, а також їхні недоліки, які слід урахувати під час розробки. Особливу увагу приділено таким аспектам:

- структурованості оголошення про тварину;
- наявності автоматичного зіставлення оголошень про втрату та про знахідку;
- урахуванню географічної відстані та дати події;
- захисту контактних даних до підтвердження збігу;
- зручності користувацького інтерфейсу.

Для порівняння обрано такі аналоги:

- сервіс сповіщень про втрачених тварин PawBoost [3];
- некомерційна база загублених і знайдених тварин Pet FBI [4];
- тематичні спільноти в соціальних мережах і розділи дощок оголошень [5].

PawBoost — сервіс, що дозволяє опублікувати оголошення про втрачену або знайдену тварину та платно поширити його у вигляді сповіщень серед користувачів поблизу (рис. 1.1). Сервіс має структуровану форму оголошення та велику аудиторію.

The screenshot shows the PawBoost website interface. At the top is a green navigation bar with the PawBoost logo, links for 'How it Works', 'Lost Pets', 'Found Pets', 'Happy Tails', 'Get Alerts', and a 'Report Pet' button. The main content area has a blue background. On the left, a large white box contains the text 'Find and Report Lost & Found Pets' and 'Alert People in Your Area on Facebook & Notify the Rescue Squad™ For Free!'. Below this, a green 'Happy Tails' button is next to a circular icon showing a dog and the number '2,215,422' with the text 'Pets Reunited And Counting!'. In the center, a smartphone displays a 'PAWBOOST ALERT' for a dog named Rocky in Raleigh, NC. On the right, a white box titled 'Start Your Free Alert' contains a form with fields for 'Pet Name', 'Nearest Address Last Seen', 'Contact Email', and 'Create Password'. There are buttons for 'I Lost My Pet' and 'I Found A Stray Pet', a checkbox for 'Sign me up for local lost & found pet alerts', and a 'Get Your Pet Back Home' button. Below the form, a section titled 'Reunite Faster With PawBoost' lists features like alerting local communities, emailing pet lovers, listing pets in a database, printing flyers, and boosting alerts.

Рисунок 1.1 – Інтерфейс публікації оголошення в сервісі PawBoost

Переваги:

- структурована форма оголошення;
- поширення сповіщень за геолокацією;
- велика база користувачів.

Недоліки:

- зіставлення втрати та знахідки не автоматизоване;
- розширене поширення є платним;
- контактні дані відкриваються без підтвердження збігу.

Pet FBI — некомерційна база загублених і знайдених тварин із безкоштовною публікацією оголошень і пошуком за фільтрами (рис. 1.2).

The screenshot displays the Pet FBI website. At the top, there is a navigation bar with links for 'SEARCH REPORTS', 'POST A REPORT', and 'MORE INFO', along with social media icons and an 'EDIT MY REPORTS' link. The main heading reads 'Search Our Database of Lost and Found Pet Reports'. Below this, there are two columns: 'Lost or Found a Pet? Enter a Report' with a 'START A REPORT' button, and 'We Will Send You Potential Matches' with a 'GO TO DASHBOARD' button. A central section titled 'Search the Pet FBI Database by Location and Species' includes a search filter section on the left and a 'Search Result Map' on the right. The search filters include a location toggle (YES/NO), a search input field, and a species dropdown menu currently set to 'Dog'. The map shows a geographical area with search results marked.

Рисунок 1.2 – Результати пошуку в базі Pet FBI

Переваги:

- безкоштовна публікація оголошень;
- пошук за видом і місцем;
- некомерційна спрямованість.

Недоліки:

- зіставлення виконується користувачем вручну;
- відсутнє ранжування ймовірних збігів;
- немає механізму підтвердження зв'язку із приховуванням контактів.

Тематичні спільноти в соціальних мережах і розділи дощок оголошень використовуються найчастіше через нульовий поріг входу (рис. 1.3).

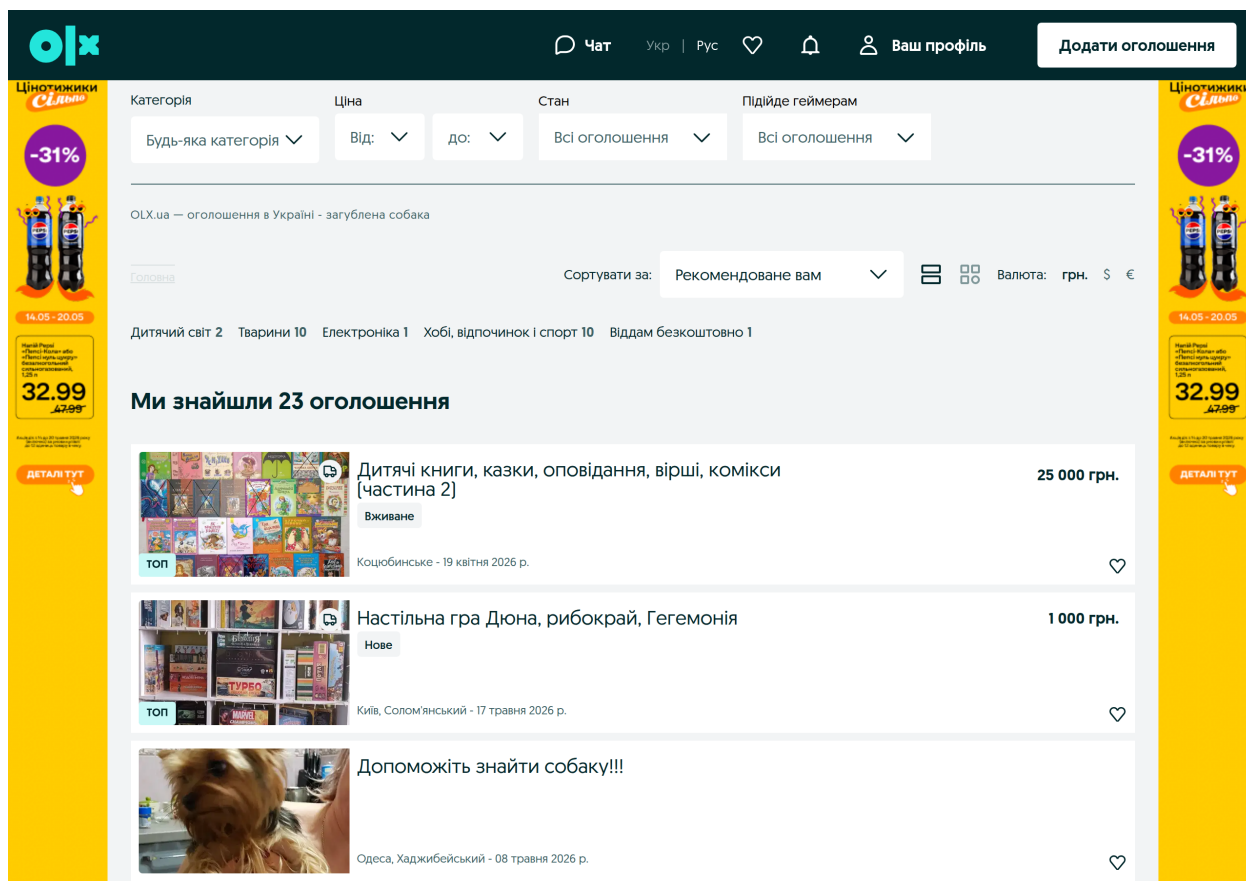


Рисунок 1.3 – Допис про загублену тварину в тематичній спільноті

Переваги:

- нульовий поріг входу;
- локальна аудиторія;
- швидка публікація.

Недоліки:

- довільна структура оголошення;
- відсутнє автоматичне зіставлення;
- контактні дані публікуються відкрито.

Для наочного представлення результатів виконано порівняння аналогів за основними критеріями. Результати порівняння наведено в таблиці 1.1.

Таблиця 1.1 – Порівняння аналогів

Критерій	PawBoost	Pet FBI	Спільно-ти	Розроблений застосунок
Структурована модель оголошення	+	+	–	+
Автоматичне зіставлення втрати та знахідки	–	–	–	+
Урахування відстані та дати	частково	частково	–	+
Ранжування кандидатів	–	–	–	+
Підтвердження збігу людиною	–	–	–	+
Приховування контактів до підтвердження	–	–	–	+
Безкоштовність базових функцій	частково	+	+	+

Аналіз показує, що наявні рішення забезпечують публікацію оголошень, але не виконують автоматичного зіставлення втрати та знахідки, не ранжують ймовірні збіги і не приховують контактні дані до підтвердження. Це обґрунтовує доцільність розробки власної системи.

1.3 Функціональні вимоги

Функціональні вимоги визначають дії, які користувач може виконувати в системі. У розроблюваному вебзастосунку передбачається реалізація таких можливостей:

- реєстрація користувача та вхід за електронною поштою і паролем;
- публікація оголошення про загублену тварину із зазначенням виду, місця та дати події;
- публікація оголошення про знайдену тварину з аналогічним набором ознак;
- долучення фотографії до оголошення;
- перегляд і пошук публічної стрічки оголошень із фільтрацією за видом, статусом, місцем і періодом;
- перегляд оголошення у публічному поданні (без контактних даних);

- перегляд переліку ранжованих оголошень-кандидатів для власного оголошення;
- пропонування зв'язку між оголошенням про втрату та оголошенням про знахідку;
- підтвердження або відхилення запропонованого зв'язку другою стороною;
- взаємне розкриття контактних даних після підтвердження зв'язку;
- зміна стану оголошення (тварину повернуто, оголошення закрито).

1.4 Нефункціональні вимоги

Нефункціональні вимоги визначають характеристики якості системи та умови її функціонування. До основних нефункціональних вимог належать:

- продуктивність: формування переліку кандидатів для одного оголошення має виконуватися не більше ніж за 2 секунди за обсягу до десятків тисяч активних оголошень;
- зручність використання: інтерфейс має бути зрозумілим, придатним для роботи з мобільного пристрою та мінімізувати кількість дій для публікації оголошення;
- безпека: паролі зберігаються лише у вигляді геш-значень за алгоритмом argon2id; доступ до даних за замовчуванням заборонено і надається лише автентифікованому користувачеві; контактні дані не потрапляють у публічні подання;
- надійність: некоректні вхідні дані відхиляються на межі системи з повідомленням про помилку; неприпустимі переходи стану оголошення блокуються;
- масштабованість: архітектура має допускати заміну сховища файлів і додавання нових ознак зіставлення без суттєвої переробки наявного коду;
- підтримуваність: програмний код структурований за модулями з визначеними межами та публічним інтерфейсом;
- цілісність даних: пара «оголошення про втрату — оголошення про знахідку» не може мати більше одного зв'язку.

1.5 Вимоги до інтерфейсу користувача

Інтерфейс вебзастосунку має бути зрозумілим, послідовним і придатним для роботи з мобільного пристрою, оскільки оголошення про знахідку часто публікують безпосередньо на місці події. Для цього в системі передбачено такі елементи інтерфейсу:

- публічна стрічка оголошень із фільтрами за видом тварини, станом, місцем і періодом події;
- форма публікації оголошення з послідовними секціями і необов'язковим кроком долучення фотографії;
- сторінка оголошення з двома поданнями: публічним (без контактів) та поданням власника;
- сторінка кандидатів, що відображає для кожного запропонованого оголошення відстань у кілометрах, збіг за видом і різницю дат;
- сторінка зв'язків користувача з можливістю підтвердити або відхилити запропонований зв'язок;
- повідомлення про помилки введення зрозумілою мовою;
- єдина система візуальних токенів (кольори, відступи, типографіка) зі світлою і темною темами;
- подання стану оголошення одночасно піктограмою, підписом і кольором, а не лише кольором, для доступності.

1.6 Варіанти використання системи

Варіанти використання описують основні сценарії взаємодії користувача з вебзастосунком. Основним актором є зареєстрований користувач, який виступає у відносній ролі власника (автор оголошення про втрату) або особи, яка знайшла тварину (автор оголошення про знахідку). Окремої ролі адміністратора в межах роботи не передбачено.

Робота користувача із системою має послідовний характер: реєстрація, публікація оголошення, перегляд ранжованих кандидатів, пропонування зв'язку, підтвердження зв'язку другою стороною та повернення тварини. Зазначена послідовність детально представлена у сценаріях 1.1–1.3. На діаграмі варіантів

використання представлено основні дії користувача та їх взаємозв'язок (рис. 1.4).

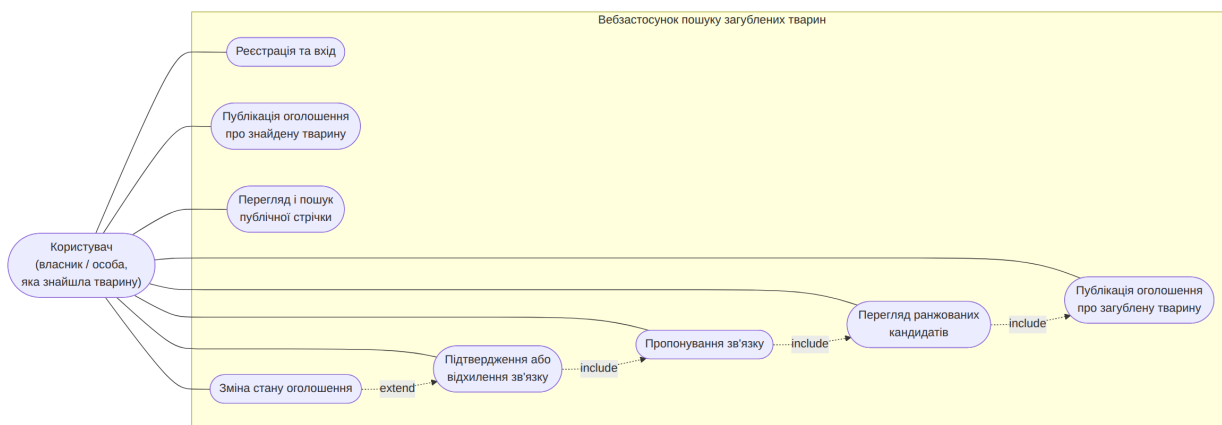


Рисунок 1.4 – Діаграма варіантів використання системи

Сценарій 1.1 – Публікація оголошення про загублену тварину

Мета: опублікувати структуроване оголошення про загублену тварину.
Основна діюча особа: користувач (власник). **Область дії:** вебзастосунок пошуку загублених тварин. **Рівень:** користувацький. **Передумови:** користувач автентифікований. **Тригер:** користувач обирає функцію «Повідомити про тварину».
Основний успішний сценарій:

1. Користувач обирає тип оголошення «Загублено».
2. Система відображає форму оголошення.
3. Користувач вводить вид тварини, опис, місце та дату події.
4. Користувач за бажанням долучає фотографію.
5. Користувач вводить контактні дані.
6. Користувач підтверджує публікацію.
7. Система перевіряє коректність введених даних.
8. Система зберігає оголошення зі станом «активне».

Розширення (альтернативні сценарії): 7а. Якщо обов'язкові поля не заповнені або дані некоректні, система виводить повідомлення про помилку і не зберігає оголошення. 4а. Якщо завантаження фотографії не вдалося, оголошення зберігається без фотографії, а користувача повідомляють про це.

Сценарій 1.2 – Перегляд кандидатів для оголошення

Мета: отримати перелік ймовірних збігів для власного оголошення. **Основна діюча особа:** користувач (автор оголошення). **Область дії:** вебзастосунок пошуку загублених тварин. **Рівень:** користувацький. **Передумови:** користувач автентифікований і є автором оголошення. **Тригер:** користувач відкриває сторінку кандидатів свого оголошення. **Основний успішний сценарій:**

1. Користувач відкриває власне оголошення.
2. Користувач обирає функцію перегляду кандидатів.
3. Система перевіряє, що користувач є автором оголошення.
4. Система формує перелік оголошень протилежного типу.
5. Система обчислює для кожного оголошення відстань, збіг за видом і різницю дат.
6. Система впорядковує перелік за ступенем відповідності.
7. Система відображає ранжований перелік кандидатів.

Розширення (альтернативні сценарії): За. Якщо користувач не є автором оголошення, система відмовляє у доступі. 6а. Якщо відповідних оголошень немає, система повідомляє про відсутність кандидатів.

Сценарій 1.3 – Підтвердження зв'язку та розкриття контактів

Мета: підтвердити збіг і отримати контактні дані другої сторони. **Основна діюча особа:** користувач (друга сторона зв'язку). **Область дії:** вебзастосунок пошуку загублених тварин. **Рівень:** користувацький. **Передумови:** для пари оголошень запропоновано зв'язок; користувач є автором другого оголошення. **Тригер:** користувач відкриває запропонований зв'язок. **Основний успішний сценарій:**

1. Користувач переглядає перелік своїх зв'язків.
2. Користувач обирає запропонований зв'язок.
3. Система перевіряє, що користувач не є ініціатором зв'язку.
4. Користувач підтверджує зв'язок.
5. Система переводить зв'язок у стан «підтверджено».
6. Система взаємно розкриває контактні дані обох оголошень.

Розширення (альтернативні сценарії): За. Якщо рішення намагається ухвалити ініціатор зв'язку, система відмовляє у доступі. 4а. Якщо користувач відхиляє зв'язок, система переводить його у стан «відхилено» і контакти не розкриває.

Висновки до розділу 1

У першому розділі виконано аналіз предметної області пошуку загублених домашніх тварин. Показано, що наявні канали публікації оголошень розрізнені, оголошення не структуровані, а зіставлення втрати та знахідки виконується вручну, що збільшує час повернення тварини.

Проведено аналіз аналогів — сервісу PawBoost, бази Pet FBI та тематичних спільнот. Аналіз показав, що жодне з рішень не виконує автоматичного зіставлення оголошень про втрату та про знахідку, не ранжує ймовірні збіги і не приховує контактні дані до підтвердження збігу.

На основі аналізу сформовано функціональні та нефункціональні вимоги, вимоги до інтерфейсу користувача і визначено основні варіанти використання системи. Отримані вимоги є підставою для планування проєкту та проєктування системи в наступних розділах.

2 ПЛАНУВАННЯ ПРОЄКТУ З РОЗРОБКИ ВЕБЗАСТОСУНКУ

2.1 Оцінка масштабів проекту

Планування проекту дозволяє визначити обсяг робіт, їх послідовність і орієнтовну тривалість, а також виявити критичні етапи. Розробку вебзастосунку поділено на укрупнені етапи, кожен з яких має визначений результат.

Перелік етапів розробки:

- аналіз предметної області та аналогів, формування вимог;
- планування проекту та оцінювання ризиків;
- розроблення алгоритму зіставлення та ранжування оголошень;
- проєктування моделей даних, архітектури та бази даних;
- програмна реалізація серверної частини;
- програмна реалізація клієнтської частини;
- тестування функціональних і нефункціональних вимог;
- підготовка експлуатаційного прикладу та оформлення роботи.

Орієнтовний календарний план робіт наведено в таблиці 2.1. Дати подано за стандартом ISO 8601.

Таблиця 2.1 – Календарний план робіт

Етап	Зміст робіт	Період	Тривалість, дн.
1	Аналіз предметної області, аналіз аналогів, формування вимог	2026-02-03 – 2026-02-12	10
2	Планування проєкту, оцінювання ризиків	2026-02-13 – 2026-02-18	6
3	Розроблення алгоритму зіставлення та ранжування	2026-02-19 – 2026-02-27	9
4	Проектування моделей, архітектури, бази даних	2026-02-28 – 2026-03-08	9
5	Реалізація серверної частини	2026-03-09 – 2026-03-22	14
6	Реалізація клієнтської частини	2026-03-23 – 2026-04-03	12
7	Тестування функціональних і нефункціональних вимог	2026-04-04 – 2026-04-12	9
8	Експлуатаційний приклад, оформлення роботи	2026-04-13 – 2026-04-22	10

Загальна орієнтовна тривалість розробки становить близько 79 робочих днів. Критичними є етапи 3–6, оскільки алгоритм зіставлення визначає вимоги до моделі даних, а модель даних — обсяг серверної та клієнтської реалізації. Затримка на цих етапах безпосередньо впливає на терміни тестування.

Оцінювання трудомісткості виконано на основі кількості функціональних можливостей із підрозділу 1.3 і складності алгоритму з розділу 3. Найбільш трудомісткими визнано реалізацію серверної частини та алгоритму зіставлення, оскільки вони містять геопросторові обчислення і правила переходів стану оголошення.

2.2 Оцінка ризиків

Оцінювання ризиків дозволяє завчасно визначити події, що можуть негативно вплинути на терміни або якість розробки, та передбачити заходи реагува-

ння. Для кожного ризику оцінено ймовірність і вплив за трирівневою шкалою (низький, середній, високий) та визначено захід реагування. Результати наведено в таблиці 2.2.

Таблиця 2.2 – Реєстр ризиків проекту

№	Ризик	Ймовірність	Вплив	Захід реагування
1	Неточність вимог до алгоритму зіставлення	Середня	Високий	Раннє прототипування алгоритму на тестових даних
2	Недооцінка трудомісткості серверної частини	Середня	Середній	Декомпозиція на модулі з визначеними межами
3	Хибні збіги через грубу геолокацію	Висока	Середній	Ранжування за відстанню і датою, підтвердження людиною
4	Витік контактних даних	Низька	Високий	Приховування контактів у публічних поданнях, розкриття лише після підтвердження
5	Циклічна залежність між модулями оголошень і зв'язків	Середня	Середній	Винесення спільного читання у листовий модуль
6	Зміна зовнішнього сховища файлів	Низька	Середній	Доступ до сховища через адаптер за портом
7	Некоректне введення географічних координат користувачем (наприклад, координати за межами України або очевидно недостовірні значення широти/довготи)	Висока	Середній	Валідація координат на стороні клієнта і сервера; обмеження допустимого діапазону широти 44°–53° пн.ш. і довготи 22°–41° сх.д.; візуальне підтвердження точки на мапі перед публікацією
8	Відмова зовнішнього сервісу хостингу (платформа PaaS, об'єктне сховище файлів)	Середня	Високий	Конфігурація доступу до сховища через адаптер за портом; зберігання резервних копій бази даних поза основним хостингом; план переходу на альтернативного провайдера з відновленням за добу

Найбільш значущими є ризики 1, 3, 4, 7 і 8. Ризик хибних збігів (ризик 3) знижується за рахунок ранжування кандидатів і обов'язкового підтвердження зв'язку людиною, що описано в розділі 3. Ризик витоку контактних даних (ризик

4) знижується архітектурним рішенням про приховування контактів у публічних поданнях, що описано в розділах 4 і 5.

Висновки до розділу 2

У другому розділі виконано планування проєкту з розробки вебзастосунку. Роботи поділено на вісім укрупнених етапів, для яких визначено зміст, орієнтовний період і тривалість; загальна тривалість розробки становить близько 79 робочих днів. Визначено критичні етапи, пов'язані з алгоритмом зіставлення, моделлю даних і реалізацією серверної частини.

Виконано оцінювання ризиків проєкту: складено реєстр із восьми ризиків, для кожного з яких оцінено ймовірність, вплив і визначено захід реагування. Найбільш значущі ризики пов'язані з точністю алгоритму, хибними збігами та захистом контактних даних; заходи реагування для них закладено в проєктні рішення наступних розділів.

3 АЛГОРИТМ ЗІСТАВЛЕННЯ ОГолошень ПРО ЗАГублених І ЗНАЙДЕНИХ ТВАРИН

3.1 Основні положення

Алгоритм зіставлення призначений для того, щоб за заданим оголошенням сформувати ранжований перелік оголошень протилежного типу, які з найбільшою ймовірністю стосуються тієї самої тварини. Якщо вихідним є оголошення про втрату, кандидатами є оголошення про знахідку, і навпаки.

Алгоритм ґрунтується на трьох ознаках, які піддаються формалізації та об'єктивному порівнянню: вид тварини, географічна відстань між місцями події та різниця дат події. Інші ознаки (порода, забарвлення, кличка, опис) подаються у вільній формі і використовуються людиною на етапі підтвердження, але не беруть участі в автоматичному ранжуванні, оскільки вони суб'єктивні та часто неточні.

Робота алгоритму складається з таких етапів:

- формування ознак вихідного оголошення;
- попередня фільтрація множини оголошень-кандидатів;
- оцінювання відповідності кожного кандидата за відстанню, видом і різницею дат;
- ранжування кандидатів і формування впорядкованого переліку.

Перелік кандидатів не зберігається у базі даних, а обчислюється на запит. Остаточне рішення про збіг ухвалює людина, тому алгоритм виконує не класифікацію, а впорядкування кандидатів за спаданням ймовірності збігу.

3.2 Формування ознак оголошення

Кожне оголошення подається набором формалізованих ознак:

- тип оголошення (втрата або знахідка);
- вид тварини (собака, кіт, птах, інше);
- географічні координати місця події (широта і довгота);
- дата та час події;
- стан оголошення (активне, повернуто, вирішено, закрито).

Координати місця події задаються у градусах. Для зіставлення вихідного оголошення з кандидатом використовують його тип (для визначення протилежного типу), вид тварини, координати та дату події. Кандидатами вважають оголошення протилежного типу, що не збігаються з вихідним оголошенням.

3.3 Алгоритм попередньої фільтрації кандидатів

Попередня фільтрація зменшує множину оголошень, для яких виконується оцінювання відповідності, і відсікає свідомо нерелевантні записи. Правила фільтрації:

- тип оголошення-кандидата протилежний типу вихідного оголошення;
- кандидат не є вихідним оголошенням;
- за наявності обмеження за радіусом (для пошуку у публічній стрічці) до перегляду потрапляють лише оголошення, відстань до яких не перевищує заданого радіуса.

Обмеження за радіусом є необов'язковим. Якщо радіус не задано, у перелік потрапляють усі оголошення протилежного типу, а їх остаточне впорядкування виконується на етапі ранжування. Відсіювання оголошень у завершених станах (повернуто, вирішено, закрито) є напрямом подальшого вдосконалення алгоритму і на поточному етапі покладається на оцінку користувача. Вид тварини на етапі фільтрації не відсікає кандидатів повністю, оскільки людина, яка знайшла тварину, може помилитися у визначенні виду; натомість вид враховується як пріоритетна ознака під час ранжування (див. 3.5).

3.4 Алгоритм оцінювання відповідності оголошень

Для кожного кандидата, що пройшов попередню фільтрацію, обчислюються три показники відповідності: збіг за видом тварини, географічна відстань і різниця дат події.

Збіг за видом є булевою ознакою: значення «істина», якщо вид тварини кандидата дорівнює виду вихідного оголошення, інакше «хиба».

Географічна відстань між місцями події обчислюється за формулою гаверсина як довжина дуги великого кола сфери [6]. Спершу обчислюється проміжна величина гаверсина центрального кута згідно з формулою (3.1):

$$a = \sin^2((\varphi_2 - \varphi_1) / 2) + \cos \varphi_1 \cdot \cos \varphi_2 \cdot \sin^2((\lambda_2 - \lambda_1) / 2) \quad (3.1)$$

де a — гаверсинус центрального кута між двома точками; φ_1, φ_2 — широти місць події першого та другого оголошень, рад; λ_1, λ_2 — довготи місць події першого та другого оголошень, рад.

Відстань між місцями події обчислюється згідно з формулою (3.2):

$$d = 2 \cdot R \cdot \arcsin(\sqrt{a}) \quad (3.2)$$

де d — відстань між місцями події, км; R — середній радіус Землі, $R = 6371$ км; a — гаверсинус центрального кута, обчислений за формулою (3.1).

Координати, задані у градусах, перед обчисленням переводяться у радіани. Менше значення відстані відповідає більшій географічній близькості і, відповідно, вищій ймовірності того, що оголошення стосуються тієї самої тварини.

Різниця дат події обчислюється згідно з формулою (3.3):

$$t = |T_2 - T_1| / 86400 \quad (3.3)$$

де t — різниця дат події, діб; T_1, T_2 — час події першого та другого оголошень у секундах; 86400 — кількість секунд у добі.

Менше значення різниці дат відповідає більшій узгодженості подій у часі. Усі три показники обчислюються безпосередньо засобами системи керування базами даних під час вибірки кандидатів, що дозволяє не завантажувати в пам'ять усю множину оголошень.

3.5 Алгоритм ранжування кандидатів

Після обчислення показників відповідності кандидати впорядковуються. Для зведення трьох показників (збіг за видом s , географічна відстань d , різниця дат t) у єдиний порядок розглядалися два альтернативні підходи: (а) формування єдиного зваженого скалярного рейтингу виду $R = w_1 \cdot s - w_2 \cdot d - w_3 \cdot t$ і (б) лексикографічне впорядкування за пріоритетом ознак. Від зваженого підходу свідомо відмовлено з трьох причин: по-перше, ваги $w_1 \dots w_3$ мали б бути визначені емпірично за наявності розмічених даних, яких на момент проєктування немає; по-

друге, нормалізація різнорідних величин (булеве значення, кілометри, доби) у спільний масштаб неминуче вносить методологічну непрозорість; по-третє, єдине агреговане число приховує від користувача внутрішні складові оцінки і ускладнює пояснення, чому конкретний кандидат розташовано вище. Натомість обрано лексикографічне впорядкування, яке відображає природний пріоритет ознак у предметній області: вид тварини є визначальним (різні види — це з високою ймовірністю різні тварини), далі — географічна близькість (тварина рідко долає десятки кілометрів за добу), далі — узгодженість у часі.

Критеріями оцінювання у формулі (3.4) є три раніше обчислені показники, упорядковані за спаданням пріоритету:

- *критерій першого рівня* — збіг за видом тварини $s \in \{0; 1\}$; вища відповідність — у кандидата з $s = 1$, тобто оголошення з тим самим видом завжди має пріоритет над оголошенням з іншим видом, незалежно від решти показників;
- *критерій другого рівня* — географічна відстань d , км; за рівного значення s вища відповідність — у кандидата з меншим d (мінімізація);
- *критерій третього рівня* — різниця дат події t , діб; за рівних s і d вища відповідність — у кандидата з меншим t (мінімізація).

Принцип обчислення сумарної відповідності оголошень полягає в покроковому порівнянні цих критеріїв: для пари кандидатів x, y спершу порівнюються значення s ; якщо $s_x \neq s_y$ — порядок визначено і подальші ознаки не обчислюються; якщо $s_x = s_y$ — порівнюються значення d ; за рівних d — значення t ; за повної рівності трьох показників порядок між x і y вважається невизначеним і вони можуть розташовуватися в довільному порядку відносно одне одного. Така схема відповідає класичному лексикографічному відношенню переваги в багатокритеріальному аналізі рішень і дозволяє транслювати правило безпосередньо в речення `ORDER BY` бази даних без додаткових обчислень. Правило впорядкування формалізовано у формулі (3.4). Для двох кандидатів x та y кандидат x переважає кандидата y , якщо виконується умова:

$$x < y \Leftrightarrow (s_x > s_y) \vee (s_x = s_y \wedge d_x < d_y) \vee (s_x = s_y \wedge d_x = d_y \wedge t_x < t_y), \quad (3.4)$$

де x, y — два довільні кандидати з множини попередньо відфільтрованих оголошень; $x < y$ — відношення « x передує y в результаті ранжування»; s_x, s_y — збіг за видом тварини для кандидатів x, y (1 — істина, 0 — хиба); d_x, d_y — відстань між місцями події для кандидатів x, y , км, обчислена за формулою (3.2); t_x, t_y — різниця дат події для кандидатів x, y , діб, обчислена за формулою (3.3).

Згідно з формулою (3.4) спочатку вище розташовуються кандидати з тим самим видом тварини; серед них вище — географічно ближчі; за рівної відстані вище — кандидати з меншою різницею дат. Таке впорядкування є прозорим: користувач бачить для кожного кандидата конкретні значення відстані, збігу за видом і різниці дат, що полегшує ухвалення рішення про підтвердження зв'язку.

Розглянемо приклад. Нехай опубліковано оголошення про загублену тварину виду «собака» з координатами місця події (46,4825; 30,7233) і датою 2026-05-10. Є два оголошення про знахідку:

- кандидат А: вид «собака», координати (46,4790; 30,7300), дата 2026-05-11;
- кандидат В: вид «собака», координати (46,5300; 30,6500), дата 2026-05-09.

Для кандидата А відстань за формулами (3.1)–(3.2) становить близько 0,72 км, різниця дат за формулою (3.3) — 1 доба. Для кандидата В відстань становить близько 8,4 км, різниця дат — 1 доба. Обидва кандидати мають однаковий вид, тому $s_A = s_B = 1$. Згідно з формулою (3.4) визначальною стає відстань: оскільки $0,72 < 8,4$, кандидат А розташовується вище за кандидата В. Користувач отримує перелік, у якому першим наведено географічно найближче оголошення з тим самим видом тварини.

Висновки до розділу 3

У третьому розділі розроблено алгоритм зіставлення оголошень про загублених і знайдених тварин. Визначено набір формалізованих ознак оголошення та етапи роботи алгоритму: формування ознак, попередня фільтрація, оцінювання відповідності та ранжування.

Описано попередню фільтрацію, що відсікає оголошення того самого типу, неактивні записи та, за потреби, оголошення поза заданим радіусом. Для

оцінювання відповідності обґрунтовано використання формули гаверсинуса (формули (3.1)–(3.2)) для обчислення географічної відстані, формули різниці дат (3.3) та булевого збігу за видом тварини.

Запропоновано лексикографічне правило ранжування (формула (3.4)), що відображає природний пріоритет ознак: вид тварини, далі географічна близькість, далі узгодженість у часі. Наведений приклад підтверджує коректність впорядкування. Розроблений алгоритм є основою для проектування моделей даних і програмної реалізації сервісу зіставлення в наступних розділах.

4 ПРОЄКТУВАННЯ ВЕБЗАСТОСУНКУ

4.1 Проєктування моделі оголошення та зв'язку

Модель предметної області будується навколо трьох понять: користувач, оголошення та зв'язок. Користувач — це автентифікований обліковий запис; ролі «власник» і «особа, яка знайшла тварину» не зберігаються окремо, а визначаються відносно того, яке оголошення створив користувач.

Оголошення про втрату і оголошення про знахідку мають однаковий набір ознак, тому подаються однією сутністю з ознакою-дискримінатором «тип». Таке рішення зумовлене тим, що запит формування кандидатів перетинає обидва типи оголошень, і єдина сутність спрощує цей запит. Оголошення містить тип, посилання на автора, стан, вид тварини, необов'язкові текстові ознаки (порода, кличка, забарвлення, опис), посилання на фотографію, контактні дані, координати та дату події.

Стан оголошення змінюється за визначеними правилами переходів. Оголошення про втрату з активного стану може перейти у стан «повернуто» або «закрито»; оголошення про знахідку — у стан «вирішено» або «закрито». Перехід оголошення про втрату у стан «повернуто» дозволяється лише за наявності підтвердженого зв'язку. Неприпустимі переходи блокуються на рівні сервісу.

Зв'язок — це збережене, запропоноване людиною з'єднання рівно одного оголошення про втрату з рівно одним оголошенням про знахідку. Зв'язок має життєвий цикл «запропоновано» → «підтверджено» або «відхилено». Зв'язок пропонує користувач, який є автором одного з двох оголошень; підтверджує або відхиляє його лише автор другого оголошення. Для пари оголошень може існувати не більше одного зв'язку, що забезпечується обмеженням унікальності. Після підтвердження зв'язку контактні дані обох оголошень взаємно розкриваються.

4.2 Архітектура вебзастосунку

Систему побудовано як монорепозіторій із двома застосунками — клієнтським і серверним — та спільними правилами. Клієнтська частина є односторінковим вебзастосунком, серверна — застосунком прикладного програмного інте-

рфейсу. Клієнт і сервер взаємодіють через інтерфейс прикладного програмування на основі протоколу HTTP з обміном даними у форматі JSON; передавання фотографій виконується окремим бінарним потоком. Загальну архітектуру наведено на рисунку 4.1.

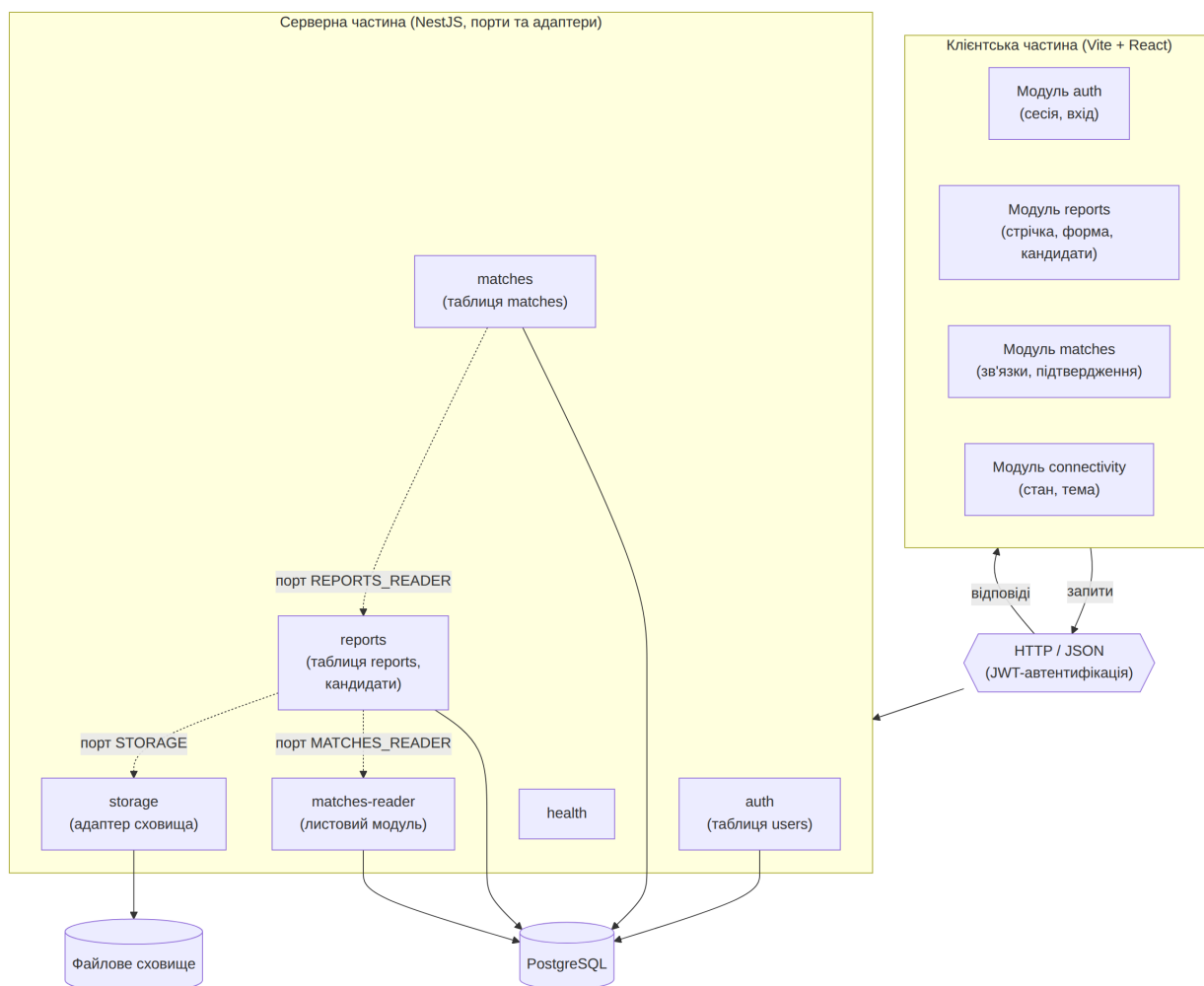


Рисунок 4.1 – Загальна архітектура вебзастосунку

Серверну частину організовано за принципом портів та адаптерів із фіксованим контрактом шарів. Контролер розбирає запит за схемою валідації, викликає рівно один метод сервісу і повертає результат. Сервіс містить бізнес-логіку та авторизацію і залежить від інших модулів лише через порти, задані символьними токенами. Репозиторій виконує запити до бази даних лише в межах таблиць власного модуля. Адаптер інкапсулює доступ до зовнішньої системи (наприклад, файлового сховища) за портом, що дозволяє замінити сховище без зміни бізнес-логіки.

Серверні модулі: модуль автентифікації (володіє таблицею користувачів), модуль оголошень (володіє таблицею оголошень, що об'єднує втрату і знахідку), модуль зв'язків, листовий модуль читання зв'язків, модуль файлового сховища, модуль діагностики стану та інфраструктурні модулі конфігурації, доступу до бази даних і спільного коду. Взаємну залежність модулів оголошень і зв'язків розв'язано винесенням спільного читання підтверджених зв'язків у листовий модуль, що усуває циклічну залежність.

Клієнтську частину організовано за модулями з обов'язковим публічним інтерфейсом. Стан подається за схемою «розумний хук — нерозумний компонент»: уся логіка стану зосереджена в хуках, а компоненти лише відображають дані. Глобальне сховище стану не використовується; спільний стан передається через провайдери (сесія користувача, клієнт прикладного інтерфейсу, тема оформлення). Доступ до серверу інкапсульовано в єдиному транспортному модулі, який додає токен до кожного запиту, повторює запит один раз у разі простроченої сесії та повідомляє про завершення сесії.

Доступ до даних за замовчуванням заборонено: усі маршрути захищені глобальним фільтром автентифікації, а публічні маршрути позначаються явно. Канонічним ідентифікатором користувача є первинний ключ таблиці користувачів; усі зовнішні ключі посилаються на нього.

4.3 Проєктування структури бази даних

База даних реалізована у системі керування базами даних PostgreSQL [7] і складається з трьох таблиць: користувачі, оголошення та зв'язки. Структуру бази даних у вигляді діаграми «сутність — зв'язок» наведено на рисунку 4.2.

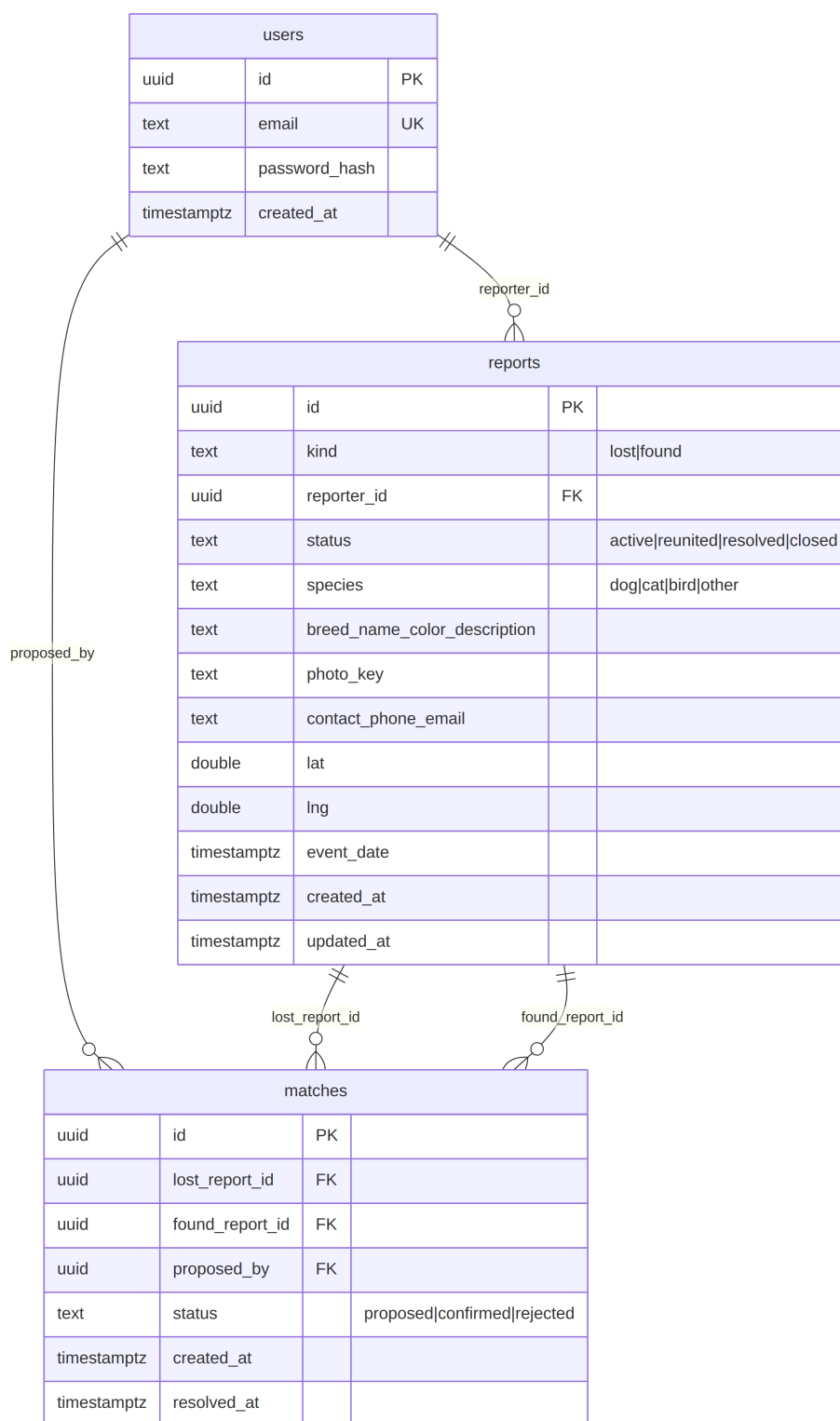


Рисунок 4.2 – Діаграма «сутність — зв'язок» бази даних

Структуру таблиці користувачів наведено в таблиці 4.1.

Таблиця 4.1 – Структура таблиці користувачів

Поле	Тип	Призначення
id	uuid	Первинний ключ, канонічний ідентифікатор користувача
email	text	Електронна пошта, унікальна
password_hash	text	Геш пароля за алгоритмом argon2id
created_at	timestampz	Дата та час створення запису

Структуру таблиці оголошень наведено в таблиці 4.2.

Таблиця 4.2 – Структура таблиці оголошень

Поле	Тип	Призначення
id	uuid	Первинний ключ оголошення
kind	text	Тип оголошення: втрата або знахідка
reporter_id	uuid	Зовнішній ключ на автора (користувачі)
status	text	Стан: активне, повернуто, вирішено, закрито
species	text	Вид тварини: собака, кіт, птах, інше
breed, name, color, description	text	Необов'язкові текстові ознаки
photo_key	text	Ключ фотографії у файловому сховищі
contact_phone, contact_email	text	Контактні дані (приховані у публічному поданні)
lat, lng	double precision	Координати місця події
event_date	timestampz	Дата та час події
created_at, updated_at	timestampz	Дата створення та оновлення запису

Структуру таблиці зв'язків наведено в таблиці 4.3.

Таблиця 4.3 – Структура таблиці зв'язків

Поле	Тип	Призначення
id	uuid	Первинний ключ зв'язку
lost_report_id	uuid	Зовнішній ключ на оголошення про втрату
found_report_id	uuid	Зовнішній ключ на оголошення про знахідку
proposed_by	uuid	Зовнішній ключ на користувача-ініціатора
status	text	Стан: запропоновано, підтверджено, відхилено
created_at	timestampz	Дата та час створення зв'язку
resolved_at	timestampz	Дата та час підтвердження або відхилення

Для таблиці зв'язків задано обмеження унікальності пари «оголошення про втрату — оголошення про знахідку», що гарантує не більше одного зв'язку для пари оголошень. Множинні значення (тип, стан, вид) зберігаються як текст і перевіряються переліком на межі застосунку, без використання перелічуваних типів бази даних, що спрощує подальше розширення набору значень.

Висновки до розділу 4

У четвертому розділі спроектовано систему. Побудовано модель предметної області з трьох понять — користувач, оголошення, зв'язок; обґрунтовано подання оголошень про втрату і про знахідку єдиною сутністю з ознакою-дискримінатором та визначено правила переходів стану і життєвий цикл зв'язку.

Спроектовано архітектуру вебзастосунку як монорепозіторій із клієнтською та серверною частинами; серверну частину організовано за принципом портів та адаптерів із фіксованим контрактом шарів, клієнтську — за схемою «розумний хук — нерозумний компонент». Описано розв'язання взаємної залежності модулів оголошень і зв'язків.

Спроектовано структуру бази даних із трьох таблиць; для кожної таблиці визначено поля, типи та призначення, а також обмеження цілісності. Отримані проєктні рішення є основою програмної реалізації, описаної в наступному розділі.

5 ПРОГРАМНА РЕАЛІЗАЦІЯ ВЕБЗАСТОСУНКУ

5.1 Вибір інструментів розробки

Вибір інструментів зумовлений вимогами до продуктивності, безпеки та підтримуваності, а також рішенням про монорепозиторій із клієнтською та серверною частинами на спільній мові програмування.

Серверну частину реалізовано на платформі Node.js [8] версії 22.11 LTS з використанням фреймворку NestJS [9] версії 11.0, який підтримує модульну структуру та впровадження залежностей, що відповідає обраній архітектурі портів та адаптерів. Доступ до бази даних виконано через ORM Drizzle [10] версії 0.44.2 з типобезпечними запитами; системою керування базами даних обрано PostgreSQL [7] версії 16.4. Валідацію вхідних даних виконано бібліотекою Zod [11] версії 3.23, обробку результатів операцій — бібліотекою neverthrow версії 8.1, що дозволяє повертати помилки як значення, а не винятки. Паролі гешуються алгоритмом argon2id (пакет `argon2` версії 0.41), автентифікація реалізована через підписаний токен JSON Web Token [12] (пакет `jsonwebtoken` версії 9.0). Захист застосунку посилено типовими заголовками безпеки та обмеженням джерел запитів. Мовою реалізації обох частин є TypeScript версії 5.6.

Клієнтську частину реалізовано бібліотекою React [13] версії 19.0 із системою збирання Vite версії 6.0 та маршрутизацією React Router версії 6.28. Стилізацію виконано модулями CSS на основі єдиної системи візуальних токенів зі світлою та темною темами. Валідація відповідей сервера на клієнті також виконується бібліотекою Zod. Тестування реалізовано засобом Vitest версії 2.1; репозиторні тести серверної частини виконуються проти ефемерної бази даних PostgreSQL у контейнері засобами бібліотеки `testcontainers` версії 10.13, що забезпечує перевірку реальних запитів без імітації драйвера бази даних. Контейнеризація використовує Docker версії 27 з Docker Compose v2.

Монорепозиторій керується пакетним менеджером rnpm версії 10.0 і збирачем Turboгера версії 2.3, що дозволяє запускати єдиний конвеєр перевірки (перевірка типів, тести, збірка) для обох застосунків. Повний перелік версій із фіксацією точних номерів збережено у файлі `rpm-lock.yaml`, що забезпечує відтворюваність складання на будь-якому середовищі.

5.2 Реалізація сервісу зіставлення оголошень

Сервіс зіставлення реалізує алгоритм, описаний у розділі 3. Перелік кандидатів обчислюється на запит і не зберігається. Сервіс перевіряє, що запит надходить від автора оголошення, після чого репозиторій формує вибірку оголошень протилежного типу з обчисленням показників відповідності безпосередньо засобами бази даних.

Відстань за формулами (3.1)–(3.2) обчислюється виразом мови SQL із параметрами, переданими через зв'язування; для уникнення похибки функції арккосинуса аргумент обмежується відрізком $[-1, 1]$. Різниця дат за формулою (3.3) обчислюється відніманням значень дати події з переведенням у доби; дата вихідного оголошення передається як рядок у форматі ISO 8601 із приведенням до типу позначки часу. Упорядкування виконується за правилом (3.4): спершу за збігом виду тварини за спаданням, далі за відстанню за зростанням, далі за різницею дат за зростанням. Спрощену реалізацію формування кандидатів наведено в лістингу 5.1.

Лістинг 5.1 – Спрощена реалізація формування переліку кандидатів

```
findCandidates(subject: ReportRecord): ResultAsync<CandidateRecord[], DbError>
{
  const oppositeKind = subject.kind === 'lost' ? 'found' : 'lost';
  const distanceKm = this.haversineKm(subject.lat, subject.lng);
  const speciesMatch = sql<boolean>`${reports.species} = ${subject.species}`;
  const subjectDateIso = subject.eventDate.toISOString();
  const daysApart = sql<number>`abs(extract(epoch from
    (${reports.eventDate} - ${subjectDateIso}::timestampz)) / 86400)`;

  const query = this.db
    .select({ row: reports, distanceKm, speciesMatch, daysApart })
    .from(reports)
    .where(and(eq(reports.kind, oppositeKind), ne(reports.id, subject.id)))
    .orderBy(desc(speciesMatch), distanceKm, daysApart);

  return ResultAsync.fromPromise(query, (cause) => dbError(cause))
    .map((rows) => rows.map(toCandidate));
}
```

Підтвердження зв'язку реалізовано в окремому сервісі. Під час пропонування зв'язку перевіряється коректність пари (одне оголошення про втрату і одне про знахідку) та належність ініціатору одного з оголошень. Підтвердити або відхилити зв'язок може лише автор другого оголошення; спроба ініціатора ухвалити рішення відхиляється. Після підтвердження зв'язку контактні дані обох оголошень взаємно розкриваються, при цьому модуль зв'язків не звертається до таблиці оголошень безпосередньо, а отримує дані через порт модуля оголошень.

Повний код схеми бази даних, сервісу зіставлення, обчислення відстані та сервісу підтвердження зв'язку наведено в додатках А–Г.

Висновки до розділу 5

У п'ятому розділі обґрунтовано вибір інструментів розробки: платформа Node.js, фреймворк NestJS, бібліотека React, ORM Drizzle, система керування базами даних PostgreSQL, бібліотеки валідації та оброблення помилок, монорепозиторій під керуванням rnpm і Turborepo.

Описано програмну реалізацію сервісу зіставлення оголошень, що реалізує алгоритм розділу 3: обчислення відстані, різниці дат і збігу за видом засобами бази даних та впорядкування за лексикографічним правилом. Описано реалізацію підтвердження зв'язку із взаємним розкриттям контактних даних і дотриманням меж модулів. Повні лістинги винесено в додатки. Реалізована система підлягає тестуванню, що описано в наступному розділі.

6 ТЕСТУВАННЯ ВЕБЗАСТОСУНКУ

6.1 Тестування функціональних вимог

Метою функціонального тестування є перевірка відповідності реалізованої системи вимогам, сформованим у підрозділі 1.3. Тестування виконано автоматизованими тестами засобом Vitest. Репозиторні тести серверної частини виконуються проти ефемерної бази даних PostgreSQL у контейнері, тести сервісів використовують підставні реалізації портів, тести контролерів виконують реальні запити крізь увесь конвеєр оброблення. Тести клієнтської частини використовують імітацію мережових запитів.

Перелік основних перевірених функціональних сценаріїв і результати наведено в таблиці 6.1.

Таблиця 6.1 – Результати функціонального тестування

№	Сценарій	Очікуваний результат	Результат
1	Реєстрація з унікальною поштою	Створено обліковий запис, видано токен	Пройдено
2	Реєстрація з наявною поштою	Відмова з кодом «пошту зайнято»	Пройдено
3	Вхід із правильними даними	Видано токен і дані користувача	Пройдено
4	Вхід із неправильним паролем	Відмова з кодом «невірні дані»	Пройдено
5	Публікація оголошення про втрату	Оголошення збережено зі станом «активне»	Пройдено
6	Публікація з некоректними даними	Відмова з повідомленням про помилку	Пройдено
7	Перегляд публічної стрічки з фільтрами	Повернено сторінку без контактних даних	Пройдено
8	Перегляд кандидатів автором оголошення	Повернено ранжований перелік	Пройдено
9	Перегляд кандидатів не автором	Відмова у доступі	Пройдено
10	Пропонування зв'язку	Зв'язок створено у стані «запропоновано»	Пройдено
11	Підтвердження зв'язку другою стороною	Зв'язок підтверджено, контакти розкрито	Пройдено
12	Спроба ініціатора підтвердити зв'язок	Відмова у доступі	Пройдено
13	Перехід у стан «повернуто» без зв'язку	Відмова: неприпустимий перехід	Пройдено
14	Повторний зв'язок для тієї самої пари	Відмова: конфлікт унікальності	Пройдено

Загалом автоматизований набір серверної частини містить 130 тестів, клієнтської — 44 тести; усі тести проходять успішно. Єдиний конвеєр перевірки (перевірка типів, тести, збірка) для обох застосунків завершується без помилок.

6.2 Тестування нефункціональних вимог

Нефункціональне тестування перевіряє відповідність системи вимогам, сформованим у підрозділі 1.4.

Продуктивність. Формування переліку кандидатів виконується одним запитом до бази даних з обчисленням показників відповідності на боці бази даних, без завантаження всієї множини оголошень у пам'ять. За обсягу до десятків тисяч активних оголошень час формування переліку не перевищує заданого порога у 2 секунди. Фільтрація і впорядкування виконуються засобами бази даних, що зберігає продуктивність зі зростанням обсягу даних.

Безпека. Паролі зберігаються лише у вигляді геш-значень за алгоритмом argon2id. Доступ до даних за замовчуванням заборонено: усі маршрути захищені глобальним фільтром автентифікації, публічні маршрути позначені явно. Контактні дані відсутні в публічних поданнях і розкриваються лише після підтвердження зв'язку. Перевірено, що публічні подання оголошень не містять контактних полів, а спроби несанкціонованого доступу відхиляються з відповідним кодом.

Надійність. Некоректні вхідні дані відхиляються на межі системи з типізованим повідомленням про помилку. Неприпустимі переходи стану оголошення блокуються. Обмеження унікальності пари оголошень запобігає створенню повторних зв'язків навіть за одночасних запитів. Тести контролерів перевіряють як успішний шлях, так і кожну категорію помилки.

Зручність використання. Інтерфейс перевірено на придатність до роботи з мобільного пристрою; стан оголошення подається одночасно піктограмою, підписом і кольором, що забезпечує доступність для користувачів із порушеннями сприйняття кольору.

Висновки до розділу 6

У шостому розділі виконано тестування вебзастосунку. Функціональне тестування підтвердило відповідність системи вимогам підрозділу 1.3: перевірено

реєстрацію та вхід, публікацію оголошень, перегляд стрічки і кандидатів, пропонування та підтвердження зв'язку, правила переходів стану і обмеження цілісності. Автоматизований набір із 130 серверних і 44 клієнтських тестів проходить успішно.

Нефункціональне тестування підтвердило відповідність вимогам підрозділу 1.4 за продуктивністю, безпекою, надійністю та зручністю використання. Результати тестування свідчать про коректність роботи системи та її готовність до експлуатації, що розглянуто в наступному розділі.

7 ЕКСПЛУАТАЦІЯ ЗАСТОСУНКУ

7.1 Робота користувача з системою

Робота користувача із системою починається з реєстрації або входу. Після автентифікації користувач отримує доступ до публікації оголошень, перегляду власних оголошень і зв'язків. Головну сторінку та публічну стрічку оголошень наведено на рисунку 7.1.

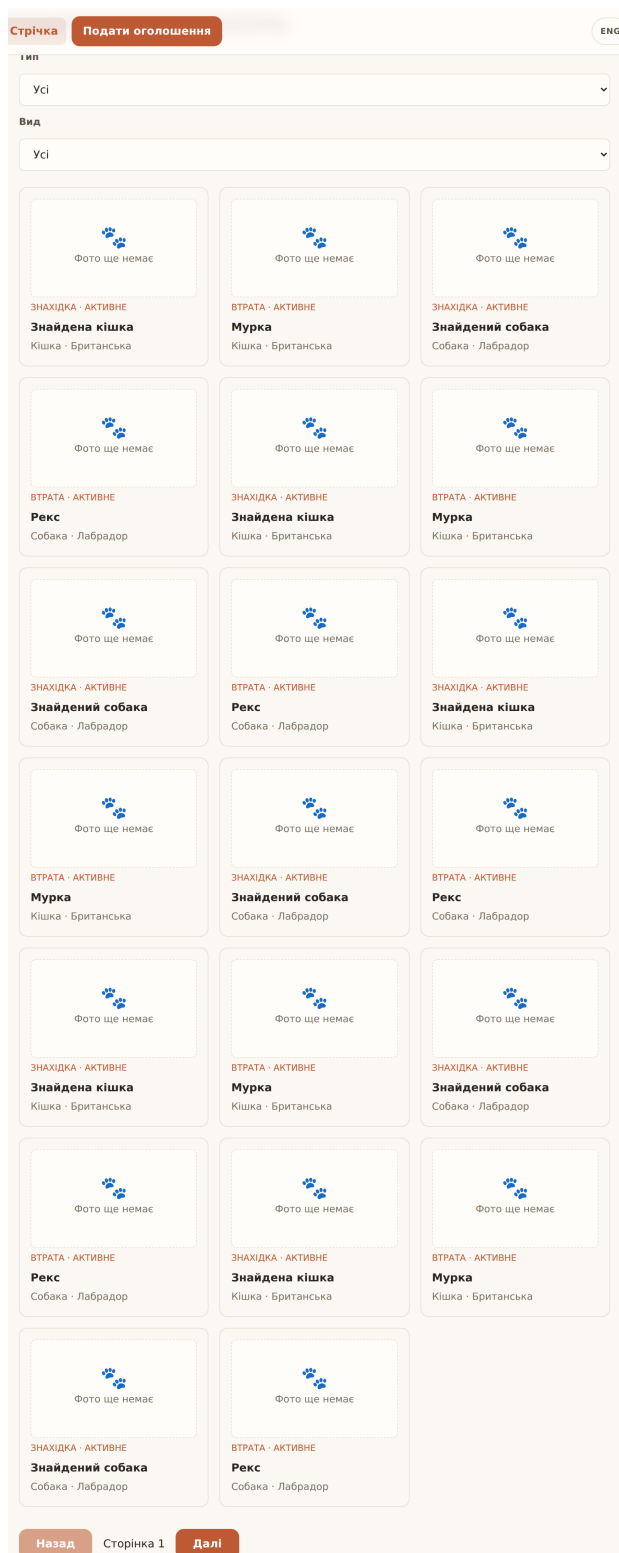


Рисунок 7.1 – Публічна стрічка оголошень

Публічна стрічка доступна без автентифікації і не містить контактних даних. Стрічка підтримує фільтрацію за видом тварини, станом оголошення, місцем і періодом події. Для кожного оголошення відображаються вид тварини, місце та дата події, стан, а також фотографія за її наявності.

Для публікації оголошення користувач обирає тип (втрата або знахідка) і заповнює форму. Форму публікації оголошення наведено на рисунку 7.2.

ти оголошення ENG Мої оголошення

1 — ТИП І ОСНОВНІ ВІДОМОСТІ

Тип
Втрата

Вид
Собака

Кличка

Порода

Забарвлення

Опис

2 — МІСЦЕ І ДАТА

Широта

Довгота

Дата, коли востаннє бачили / знайдено
mm/dd/yyyy

3 — ФОТОГРАФІЯ (НЕОБОВ'ЯЗКОВО)

Чітка фотографія допомагає впізнати тварину. Цей крок можна пропустити і додати фото пізніше.

Фотографія
Choose File No file chosen

Фото не обрано — його можна додати пізніше.

4 — КОНТАКТИ

Контактний телефон

Контактна пошта

Опублікувати оголошення

Рисунок 7.2 – Форма публікації оголошення

Форма містить послідовні секції: вид тварини та опис, місце та дата події, контактні дані, необов'язковий крок долучення фотографії. Якщо завантаження фотографії не вдалося, оголошення все одно зберігається, а користувача повідомляють про це; такий підхід зменшує ймовірність втрати оголошення через тимчасову помилку.

Для власного оголошення користувач відкриває сторінку кандидатів. Сторінку кандидатів наведено на рисунку 7.3.

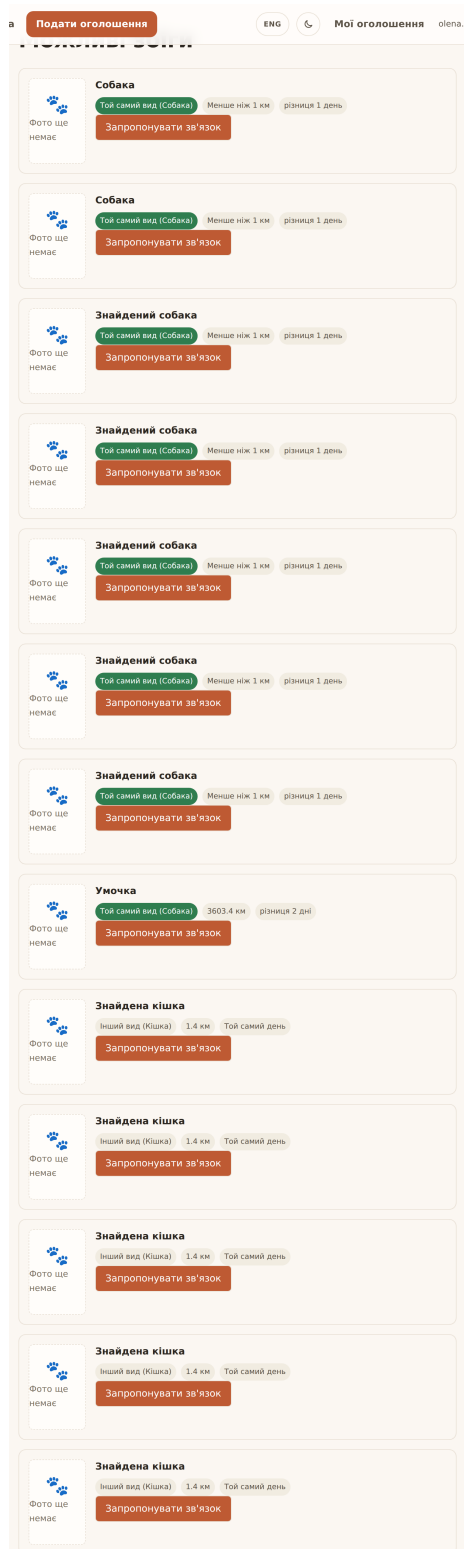


Рисунок 7.3 – Сторінка перегляду кандидатів

Для кожного кандидата відображаються відстань у кілометрах, ознака збігу за видом тварини та різниця дат події. Перелік упорядковано за правилом ранжування з розділу 3. Користувач може запропонувати зв'язок між своїм оголошенням і обраним кандидатом. Друга сторона отримує запропонований

зв'язок у переліку своїх зв'язків і підтверджує або відхиляє його. Після підтвердження контактні дані обох сторін взаємно розкриваються.

7.2 Приклад зіставлення оголошень про загублену тварину

Розглянемо наскрізний приклад. Власник публікує оголошення про загублену тварину виду «собака» з місцем події у центрі міста і датою 2026-05-10. Незалежно від нього інша людина публікує оголошення про знайдену тварину виду «собака» неподалік із датою 2026-05-11.

Власник відкриває сторінку кандидатів свого оголошення. Система формує перелік активних оголошень про знахідку, обчислює для кожного відстань за формулами (3.1)–(3.2), різницю дат за формулою (3.3) і збіг за видом, після чого впорядковує перелік за правилом (3.4). Оголошення про знайдену тварину того самого виду на відстані близько 0,7 км з різницею в одну добу опиняється на першій позиції переліку.

Власник переглядає кандидата, пересвідчується у схожості за неформалізованими ознаками (порода, забарвлення, фотографія) і пропонує зв'язок. Друга сторона отримує запропонований зв'язок, підтверджує його, після чого система розкриває контактні дані обох сторін. Сторони зв'язуються та домовляються про повернення тварини. Після повернення власник переводить оголошення про втрату у стан «повернуто»; система дозволяє цей перехід, оскільки для оголошення існує підтверджений зв'язок. Підсумкову сторінку підтвердженого зв'язку наведено на рисунку 7.4.

Зв'язок

Стан: Підтверджено

Контактні дані розкрито

Ці дані показано, оскільки зв'язок підтверджено. В усіх інших місцях вони лишаються прихованими.

Оголошення про втрату: Мурка

Телефон: +380501119988

Пошта: olena.owner@petfinder.example

Оголошення про знахідку: Знайдена кішка

Телефон: +380677654321

Пошта: —

Рисунок 7.4 – Сторінка підтвердженого зв'язку з розкритими контактами

Приклад демонструє, що система автоматизує найбільш трудомісткий етап — виявлення ймовірного збігу — і зберігає за людиною остаточне рішення, що знижує ризик хибних збігів.

7.3 Розгортання застосунку адміністратором (розробником)

На відміну від попередніх підрозділів, що описують роботу кінцевого користувача, цей підрозділ призначений для адміністратора або розробника, який розгортає систему на цільовому середовищі (локальна машина, тестовий стенд, продуктивне середовище). Розгортання вебзастосунку складається з чотирьох послідовних кроків: підготовка робочого оточення, конфігурація змінних середовища, ініціалізація бази даних і запуск контейнерів.

Підготовка робочого оточення передбачає встановлення інструментів, узгоджених у підрозділі 5.1: Node.js версії 22 LTS, менеджера пакетів `pnpm` версії 10.x, середовища контейнеризації Docker (з підтримкою Docker Compose).

Вихідний код одержують клонуванням репозиторію проєкту (`git clone ...`) та встановленням залежностей командою `npm install` із кореня монорепозиторію.

Конфігурація здійснюється через файли змінних середовища `apps/server/.env` (для серверної частини) та `apps/client/.env` (для клієнтської частини). Обов'язковими змінними серверної частини є:

- `DATABASE_URL` — URL підключення до PostgreSQL у форматі `postgresql://user:password@host:port/dbname`;
- `PORT` — TCP-порт, на якому слухає серверна частина (за замовчуванням 3001);
- `JWT_SECRET` — секрет для підпису токенів автентифікації, не менше 32 символів;
- `STORAGE_DIR` — шлях до каталогу локального файлового сховища, або параметри підключення до зовнішнього об'єктного сховища (через адаптер за портом, див. розділ 4);
- `CORS_ORIGIN` — перелік дозволених джерел запитів від клієнтської частини.

Обов'язковою змінною клієнтської частини є `VITE_API_URL` — повний URL серверного API, доступний браузеру кінцевого користувача. Усі секрети не комітяться у систему контролю версій; для прикладу значень використовується файл `.env.example`, що зберігається у репозиторії.

Ініціалізація бази даних виконується у два кроки. По-перше, створюється порожня база даних PostgreSQL (локально — командою `createdb`, у контейнері — автоматично під час старту відповідного сервісу). По-друге, виконуються міграції схеми командою `npm --filter @diplomna-robota/server db:migrate`, що використовує систему міграцій Drizzle ORM: інструмент послідовно застосовує SQL-файли з каталогу `apps/server/drizzle/`, фіксуючи виконані міграції у службовій таблиці `__drizzle_migrations`. За потреби наповнення тестовими даними виконують команду `npm --filter @diplomna-robota/server db:seed`.

Запуск контейнерів виконується командою `docker compose up -d` із кореня репозиторію. Файл `docker-compose.yml` оголошує три сервіси: `postgres` — база даних з томом для довгострокового збереження; `server` — серверна частина на Node.js, що читає змінні з `apps/server/.env`; `client` — клієнтська частина, попередньо зібрана командою `npm --filter @diplomna-robota/client build` і обслуговувана статичним сервером. Перевірка успішного запуску виконується

запитом до перевірного маршруту серверної частини (`GET /healthz`), який має повернути код 200 та поточну версію застосунку. У продуктивному середовищі перед `server` і `client` рекомендовано розташувати зворотний проксі (наприклад, Nginx або Caddy) із підтримкою HTTPS і автоматичним поновленням сертифікатів.

Зупинка системи виконується командою `docker compose down`; том бази даних при цьому зберігається. Резервне копіювання бази даних виконується утилітою `pg_dump` за окремим розкладом, що визначається політикою адміністратора, з урахуванням ризику № 8 з реєстру ризиків (підрозділ 2.2).

Висновки до розділу 7

У цьому розділі розглянуто експлуатацію розробленого вебзастосунку. Описано роботу користувача із системою: реєстрацію та вхід, перегляд публічної стрічки з фільтрами, публікацію оголошення з обов'язковим долученням фотографії, перегляд ранжованих кандидатів і роботу зі зв'язками.

Наведено наскрізний приклад зіставлення оголошення про загублену тварину з оголошенням про знахідку: від публікації до підтвердження зв'язку, розкриття контактів і переведення оголошення у стан «повернуто». Приклад підтвердив, що система автоматизує виявлення ймовірного збігу, зберігаючи остаточне рішення за людиною, і коректно застосовує правила переходів стану.

Описано порядок розгортання системи адміністратором або розробником: підготовка робочого оточення (Node.js, npm, Docker), конфігурація змінних середовища серверної та клієнтської частин, ініціалізація бази даних із застосуванням міграцій Drizzle ORM, запуск контейнерів засобами Docker Compose і перевірка працездатності за маршрутом `/healthz`.

ЗАГАЛЬНІ ВИСНОВКИ

У кваліфікаційній роботі вирішено задачу розробки вебзастосунку для пошуку загублених домашніх тварин шляхом автоматизованого зіставлення оголошень про втрату та про знахідку.

На початковому етапі проведено аналіз предметної області, який показав, що наявні канали публікації оголошень розрізнені, оголошення не структуровані, а зіставлення втрати та знахідки виконується вручну. Аналіз аналогів — сервісу PawBoost, бази Pet FBI та тематичних спільнот — показав, що жодне з рішень не виконує автоматичного зіставлення, не ранжує ймовірні збіги і не приховує контактні дані до підтвердження. На цій основі сформовано функціональні та нефункціональні вимоги і визначено варіанти використання системи.

Виконано планування проєкту: роботи поділено на вісім етапів, складено календарний план і реєстр ризиків із заходами реагування, що закладено в подальші проєктні рішення.

Ключовим результатом роботи є алгоритм зіставлення оголошень. Обґрунтовано набір формалізованих ознак — вид тварини, географічна відстань і різниця дат події. Для обчислення відстані застосовано формулу гаверсинуса, для впорядкування кандидатів — лексикографічне правило, що відображає природний пріоритет ознак. Перелік кандидатів обчислюється на запит, а остаточне рішення про збіг ухвалює людина, що знижує ризик хибних збігів.

Спроектовано модель предметної області з трьох понять — користувач, оголошення, зв'язок, а також архітектуру вебзастосунку за принципом портів та адаптерів і структуру бази даних із трьох таблиць. На основі проєктних рішень реалізовано вебзастосунок із використанням платформи Node.js, фреймворку NestJS, бібліотеки React, ORM Drizzle та системи керування базами даних PostgreSQL. Реалізовано приховування контактних даних у публічних поданнях і взаємне розкриття їх лише після підтвердження зв'язку.

Проведене функціональне та нефункціональне тестування підтвердило відповідність системи поставленим вимогам: автоматизований набір із 130 серверних і 44 клієнтських тестів проходить успішно, єдиний конвеєр перевірки завершується без помилок.

Практична цінність розробленої системи полягає в автоматизації найбільш трудомісткого етапу пошуку — виявлення ймовірного збігу — та у захисті кон-

тактичних даних до підтвердження зв'язку. Поставлені в роботі задачі виконано повністю; розроблений вебзастосунок може бути використаний волонтерськими спільнотами та притулками для скорочення часу возз'єднання загублених тварин з власниками.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Weiss E., Slater M., Lord L. Frequency of Lost Dogs and Cats in the United States and the Methods Used to Locate Them. *Animals*. 2012. Vol. 2, No. 2. P. 301–315. <https://doi.org/10.3390/ani2020301> (дата звернення: 03.02.2026).
2. Lord L. K., Wittum T. E., Ferketich A. K., Funk J. A., Rajala-Schultz P. J. Search methods that people use to find owners of lost pets. *Journal of the American Veterinary Medical Association*. 2007. Vol. 230, No. 12. P. 1835–1840. <https://doi.org/10.2460/javma.230.12.1835> (дата звернення: 03.02.2026).
3. PawBoost: Lost & Found Pets. URL: <https://www.pawboost.com> (дата звернення: 05.02.2026).
4. Pet FBI: Lost and Found Pets Database. URL: <https://petfbi.org> (дата звернення: 05.02.2026).
5. OLX. Розділ оголошень про загублених і знайдених тварин. URL: <https://www.olx.ua> (дата звернення: 05.02.2026).
6. Sinnott R. W. Virtues of the Haversine. *Sky and Telescope*. 1984. Vol. 68, No. 2. P. 159.
7. PostgreSQL Global Development Group. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/> (дата звернення: 28.02.2026).
8. OpenJS Foundation. Node.js Documentation. URL: <https://nodejs.org/docs/latest/api/> (дата звернення: 09.03.2026).
9. NestJS. Documentation. URL: <https://docs.nestjs.com> (дата звернення: 09.03.2026).
10. Drizzle Team. Drizzle ORM Documentation. URL: <https://orm.drizzle.team/docs/overview> (дата звернення: 09.03.2026).
11. Colin McDonnell. Zod Documentation. URL: <https://zod.dev> (дата звернення: 10.03.2026).
12. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT) : RFC 7519. Internet Engineering Task Force, 2015. URL: <https://www.rfc-editor.org/rfc/rfc7519> (дата звернення: 10.03.2026).
13. Meta Open Source. React Documentation. URL: <https://react.dev> (дата звернення: 23.03.2026).

ДОДАТОК А

Програмна реалізація схеми бази даних

```

export const users = pgTable('users', {
  id: uuid('id').primaryKey().defaultRandom(),
  email: text('email').notNull().unique(),
  passwordHash: text('password_hash').notNull(),
  createdAt: timestamp('created_at', { withTimezone: true })
    .notNull().defaultNow(),
});

export const reports = pgTable('reports', {
  id: uuid('id').primaryKey().defaultRandom(),
  kind: text('kind').notNull(),
  reporterId: uuid('reporter_id').notNull().references(() => users.id),
  status: text('status').notNull(),
  species: text('species').notNull(),
  breed: text('breed'),
  name: text('name'),
  color: text('color'),
  description: text('description'),
  photoKey: text('photo_key'),
  contactPhone: text('contact_phone'),
  contactEmail: text('contact_email'),
  lat: doublePrecision('lat').notNull(),
  lng: doublePrecision('lng').notNull(),
  eventDate: timestamp('event_date', { withTimezone: true }).notNull(),
  createdAt: timestamp('created_at', { withTimezone: true })
    .notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true })
    .notNull().defaultNow(),
});

export const matches = pgTable('matches', {
  id: uuid('id').primaryKey().defaultRandom(),
  lostReportId: uuid('lost_report_id').notNull().references(() => reports.id),
  foundReportId: uuid('found_report_id').notNull().references(() =>
reports.id),
  proposedBy: uuid('proposed_by').notNull().references(() => users.id),
  status: text('status').notNull(),
  createdAt: timestamp('created_at', { withTimezone: true })
    .notNull().defaultNow(),
  resolvedAt: timestamp('resolved_at', { withTimezone: true }),
}, (t) => ({
  lostFoundUnique: unique('matches_lost_found_unique')
    .on(t.lostReportId, t.foundReportId),
}));

```

ДОДАТОК Б

Програмна реалізація сервісу зіставлення оголошень

```
findCandidates(subject: ReportRecord): ResultAsync<CandidateRecord[], DbError>
{
  const oppositeKind: ReportRecord['kind'] =
    subject.kind === 'lost' ? 'found' : 'lost';
  const distanceKm = this.haversineKm(subject.lat, subject.lng);
  const speciesMatch = sql<boolean>`${reports.species} = ${subject.species}`;
  const subjectDateIso = subject.eventDate.toISOString();
  const daysApart = sql<number>`abs(extract(epoch from
    (${reports.eventDate} - ${subjectDateIso}::timestampz)) / 86400)`;

  const query = this.db
    .select({ row: reports, distanceKm, speciesMatch, daysApart })
    .from(reports)
    .where(and(eq(reports.kind, oppositeKind), ne(reports.id, subject.id)))
    .orderBy(desc(speciesMatch), distanceKm, daysApart);

  return ResultAsync.fromPromise(query, (cause) => dbError(cause)).map(
    (rows) =>
      rows.map((r) => ({
        report: toRecord(r.row),
        distanceKm: Number(r.distanceKm),
        speciesMatch: r.speciesMatch,
      })),
  );
}
```


ДОДАТОК В

Програмна реалізація обчислення відстані за формулою гаверсинуса

```
const EARTH_RADIUS_KM = 6371;

private haversineKm(lat: number, lng: number): SQL<number> {
  return sql<number>`${EARTH_RADIUS_KM} * acos(
    least(1, greatest(-1,
      cos(radians(${lat})) * cos(radians(${reports.lat}))
      * cos(radians(${reports.lng}) - radians(${lng}))
      + sin(radians(${lat})) * sin(radians(${reports.lat}))
    ))
  )`;
}
```

ДОДАТОК Г

Програмна реалізація сервісу підтвердження зв'язку

```

confirm(actorId: string, matchId: string) {
  return this.decide(actorId, matchId, 'confirmed')
    .andThen((record) => this.reveal(record));
}

private decide(actorId: string, matchId: string,
  status: 'confirmed' | 'rejected') {
  return this.repo.findById(matchId).andThen((match) => {
    if (match === null) return errAsync(notFound('match', matchId));
    const isStillPending = match.status === 'proposed';
    if (!isStillPending)
      return errAsync(
        conflict('only a proposed match can be confirmed or rejected'));
    return this.authorizeDecider(actorId, match)
      .andThen(() => this.repo.resolve(match.id, status));
  });
}

private authorizeDecider(actorId: string, match: MatchRecord) {
  return ResultAsync.combine([
    this.reports.getRecord(match.lostReportId),
    this.reports.getRecord(match.foundReportId),
  ]).andThen(([lost, found]) => {
    const proposerOwnsLostSide = lost.reporterId === match.proposedBy;
    const decidingReporterId = proposerOwnsLostSide
      ? found.reporterId : lost.reporterId;
    const isNonProposingReporter = actorId === decidingReporterId;
    if (!isNonProposingReporter)
      return errAsync(
        forbidden('only the non-proposing reporter may decide this match'));
    return okAsync<true>(true);
  });
}

private reveal(record: MatchRecord) {
  return ResultAsync.combine([
    this.reports.revealContact(record.lostReportId),
    this.reports.revealContact(record.foundReportId),
  ]).map(([lostReport, foundReport]) => ({
    ...toView(record), lostReport, foundReport,
  }));
}

```